

Vibe Coding

CSE 490 A2 · Lecture 1 · Intro + Prompt to App



The Instructional Team



Steve Seitz
PROFESSOR



Vinamra Agarwal
TA



Ella Cao
TA



Prabhgun Basi
TA



Arian Shamaei
TA



Aditya Kumar
TA

Readings

- [Large Language Models from scratch](#) – 8 min
- [Large Language Models: Part 2](#) – 7 min
- [Welcome to Lovable](#) – 4 min
- Optional: [Intro to LLMs \(Karpathy\)](#) – 60 min

LLM Takeaways (Karpathy)

- "Zip file of the Internet"
 - 100x, lossy compression
 - > 6 months old
- Hallucinates, jailbreaks
- Tool calling adds power
- Trained to think (RL)
- Context costs

The Evolution of Coding

```
xor eax, eax  
add eax, [esi]
```

Assembly



1949

```
total = sum(nums)
```

Python



1991

“add up the numbers”

English



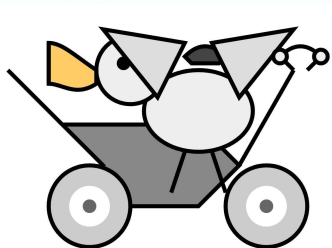
2023



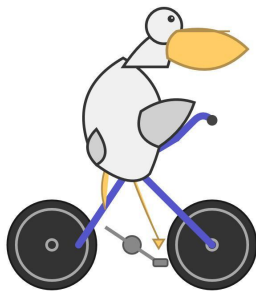
None of this is vibe coding yet

Now with Pictures

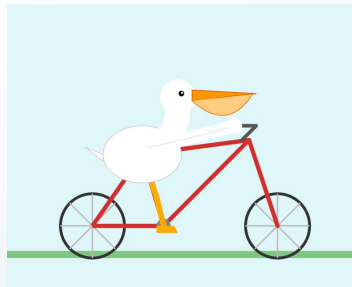
Prompt: *"Generate an SVG of a pelican riding a bicycle"*



Dec 2024
Gemini 2.0 Flash Thinking



Mar 2025
Gemini 2.5 Pro



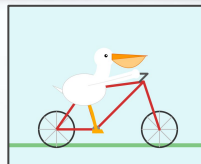
Nov 2025
Gemini 3 Pro



Sep 2026
Claude Fable 5.1

simonwillison.net, the pelican-riding-a-bicycle series

What the Model Actually Wrote (SVG)



```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 500 400">
  <!-- Background -->
  <rect width="100%" height="100%" fill="#e0f7fa" />
  <line x1="0" y1="350" x2="500" y2="350" stroke="#81c784" stroke-width="10" />

  <!-- Bicycle -->
  <g id="bicycle" transform="translate(0, 10)">
    <!-- Wheels -->
    <g stroke="#333" stroke-width="4" fill="none">
      <circle cx="120" cy="300" r="45" />
      <circle cx="380" cy="300" r="45" />
    <!-- Spokes -->
    <g stroke="#bbb" stroke-width="2">
      <path d="M120 300 145 0 M120 300 145 M120 300 10 -45" />
      <path d="M120 300 132 32 M120 300 1-32 M120 300 1-32 32 M120 300 132 -32" />
      <path d="M380 300 145 0 M380 300 145 M380 300 10 -45" />
      <path d="M380 300 132 32 M380 300 1-32 M380 300 1-32 32 M380 300 132 -32" />
    </g>
  </g>

  <!-- Frame -->
  <path d="M120 300 L220 300 L190 200 L120 300" fill="none" stroke="#d32f2f" stroke-width="6"
stroke-linejoin="round" />
  <path d="M220 300 L340 180" fill="none" stroke="#d32f2f" stroke-width="6" stroke-linecap="round" />
  <path d="M190 200 L340 180" fill="none" stroke="#d32f2f" stroke-width="6" stroke-linecap="round" />

  <!-- Fork and Handlebars -->
  <path d="M340 180 L380 300" stroke="#d32f2f" stroke-width="6" stroke-linecap="round" />
  <path d="M380 180 L350 160 L320 160" fill="none" stroke="#555" stroke-width="5"
stroke-linecap="round" stroke-linejoin="round" />

  <!-- Seat -->
  <ellipse cx="180" cy="195" rx="25" ry="8" fill="#333" />

  <!-- Crank/Chaining -->
  <circle cx="220" cy="300" r="10" fill="#888" />
</g>

  <!-- Pelican -->
  <g id="pelican" transform="translate(0, 10)">
    <!-- Back Leg (Pedaling) -->
    <path d="M200 230 L210 260 L220 300" fill="none" stroke="#ffab00" stroke-width="8"
stroke-linecap="round" />
    <path d="M210 300 L235 300 L240 310 L210 310 Z" fill="#ffab00" /> <!-- Webbed Foot -->

    <!-- Body -->
    <ellipse cx="190" cy="200" rx="55" ry="40" fill="#fff" stroke="#ccc" stroke-width="1" />

    <!-- Tail Feathers -->
    <path d="M135 200 L110 190 L135 210 Z" fill="#fff" stroke="#ccc" stroke-width="1" />

    <!-- Neck -->
    <path d="M220 180 Q240 170 240 130" fill="none" stroke="#fff" stroke-width="35"
stroke-linecap="round" />

    <!-- Head -->
    <circle cx="240" cy="120" r="25" fill="#fff" />

    <!-- The Beak (Upper) -->
    <path d="M260 110 L350 115 L260 125 Z" fill="#ff9800" stroke="#e65100" stroke-width="1" />

    <!-- The Pouch (Lower Beak) -->
    <path d="M260 125 Q310 170 345 116" fill="#ffcc80" stroke="#ef6c00" stroke-width="1" />

    <!-- Eye -->
    <circle cx="245" cy="115" r="4" fill="black" />
    <circle cx="246" cy="114" r="1.5" fill="white" />

    <!-- Wing (Reaching for handlebars) -->
    <path d="M200 190 Q250 180 320 160" fill="none" stroke="#fff" stroke-width="18"
stroke-linecap="round" />
    <path d="M200 190 Q250 180 320 160" fill="none" stroke="#eee" stroke-width="2" /> <!-- Wing detail
line -->

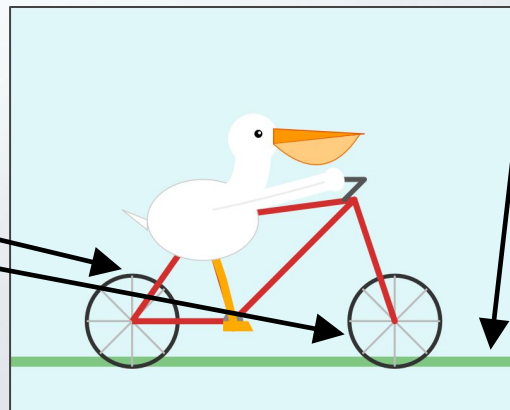
    <!-- Wing Tip/Feathers on Handlebar -->
    <ellipse cx="320" cy="160" rx="10" ry="12" fill="#fff" transform="rotate(-20 320 160)" />
  </g>
</svg>
```

Gemini 3 Pro, November 2025, all 72 lines: simonwillison.net, the pelican-riding-a-bicycle series

Zoom In: The First Lines

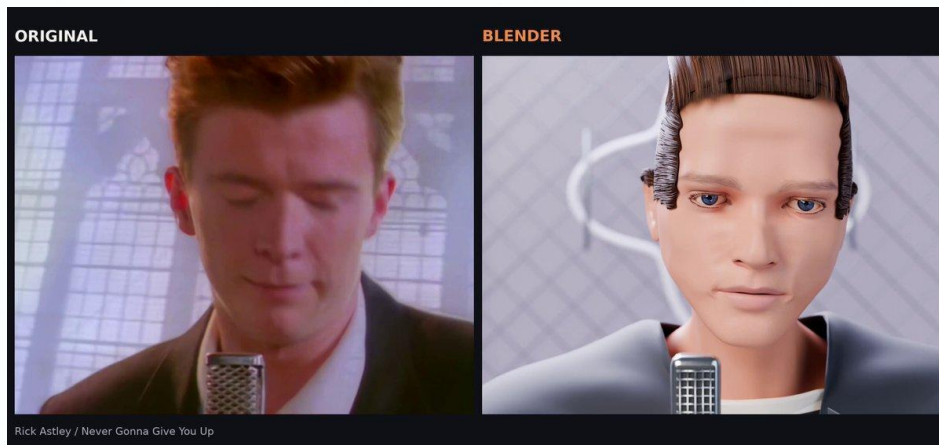
```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 500 400">
  <!-- Background -->
  <rect width="100%" height="100%" fill="#e0f7fa" />
  <line x1="0" y1="350" x2="500" y2="350" stroke="#81c784" stroke-width="10" />

  <!-- Bicycle -->
  <g id="bicycle" transform="translate(0, 10)">
    <!-- Wheels -->
    <g stroke="#333" stroke-width="4" fill="none">
      <circle cx="120" cy="300" r="45" />
      <circle cx="380" cy="300" r="45" />
```



[Gemini 3 Pro, November 2025, lines 1-11 of 72: simonwillison.net, the pelican-riding-a-bicycle series](#)

Now with Video



VB Srivastav (OpenAI, @reach_vb) on X, 6 September 2026

Prompt: "create a side by side video of rickroll & a version created w/ blender, use a subagents to verify your output as you go. Use web search to get necessary assets for the task."

Nobody Agrees What to Call It

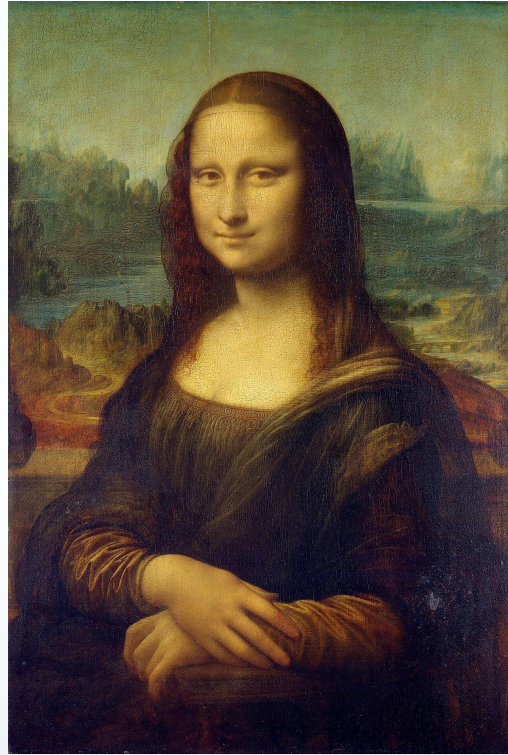
- Vibe coding – Karpathy, February 2025
- Agentic engineering – Karpathy, 2026
- Loop engineering – Osmani, 2026
- CHOP – Yegge, 2024 (chat-oriented programming)

Superpowers



<https://www.aventon.com/products/level-4-rec-ebike>

Iteration



Leonardo da Vinci, c. 1503–1517 · Louvre · public domain, via Wikimedia Commons



Adaptation



"Lens of Time: Slime Lapse", bioGraphic, on YouTube

The Course



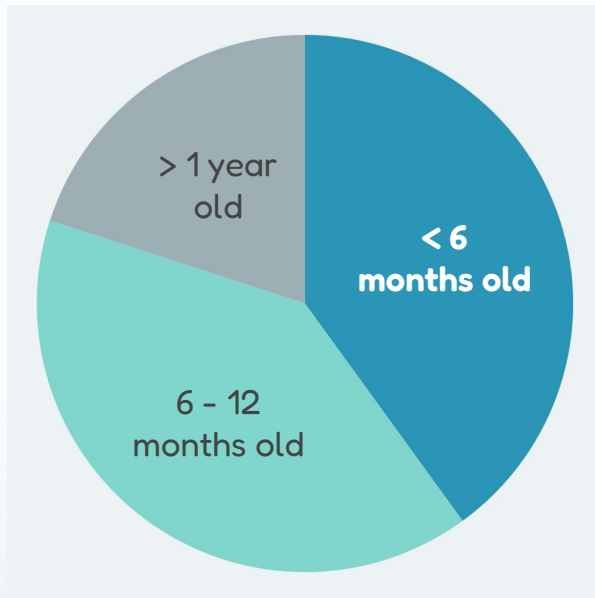
What This Course Is For

- Become a proficient AI software engineer
- Teach what matters as of today
- Learn how these systems work
- Build real applications with AI

Vibe Coding: The Syllabus

- Week 1: Prompt to App
- Week 2: The Prompt-Run-Refine Loop
- Week 3: The Agent Harness
- Week 4: Steering an Agent
- Week 5: MCP and Agent Governance
- Week 6: Working in Code You Did Not Write
- Week 7: Multi-Agent Orchestration
- Week 8: Local Models
- Week 9: Evals and Goal Loops
- Week 10: Deploy Behind CI

How New Is Any of This?



80% of this
class didn't
exist last
year



Course Innovations

- Learn by building
- The *course* is vibe-coded
- *Extreme* freshness
- Autograding



Course Project

Build your own version of Claude Code.



Assignments

- 10 projects, started in class
- Occasional surveys
- Auto-graded
 - turn in code and prompt history (keystrokes!)
 - grade *how you created the project*

Tools: “Officially” Supported

- Purple: chat-based
- Amazon Bedrock: many models, UW contract
 - Covered by FERPA, so course data is safe
- Claude Code on lab machines*, or your laptop

Not UW Supported

- The best tools are not UW-supported
 - Lovable, Claude Pro/Max, Codex
- We recommend you use them anyway
 - Not required
 - Backup options available if you prefer

Today



The Tools, in Three Groups

Builders

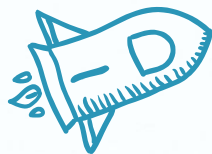
Prompt in, deployed app out. Bolt, Lovable, Replit, v0. You use one today.

In-editor

You stay in the editor and read the diffs. Cursor, Copilot.

Agents

It writes, runs, and fixes in a loop. Claude Code, Codex, Gemini CLI.



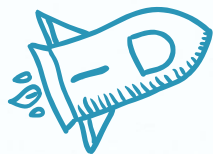
PROMPT → APP

You describe an app in plain English.
Forty minutes later the app is live!



How Do AI Coding Tools Work?



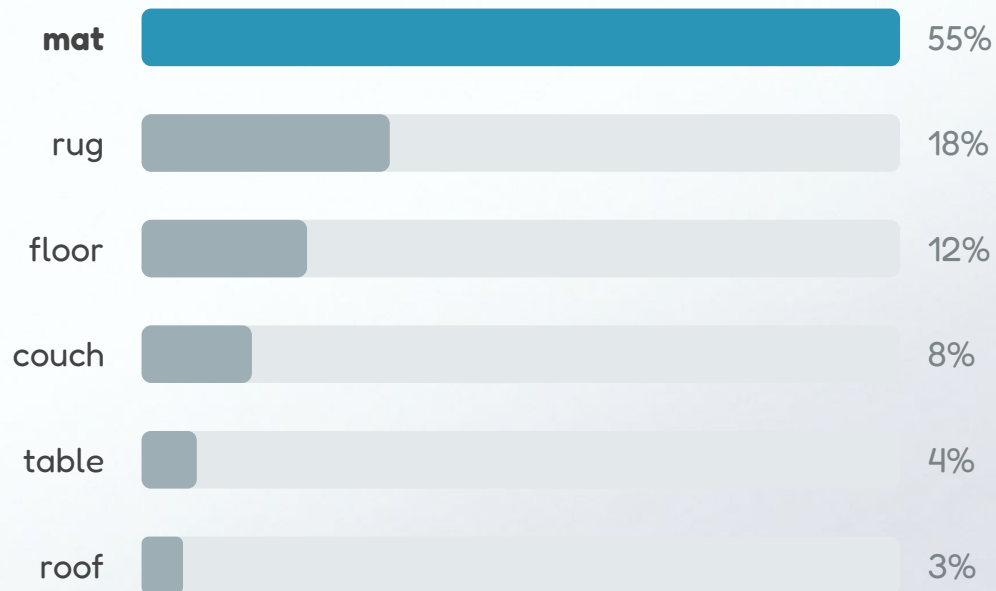


THEY PREDICT

What word comes next:
"the cat sat on the ..."



The Cat Sat on the ____



Autoregressive: The Output Feeds Back In

```
1  remove an item from an array
2  remove an item from an array const
3  remove an item from an array const result
4  remove an item from an array const result =
5  remove an item from an array const result = arr.filter(
```

Same Prompt, Run Twice

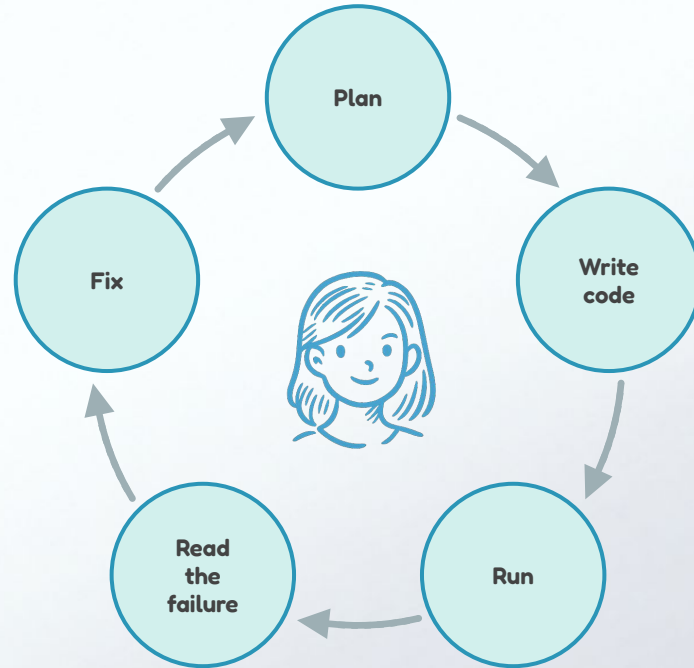
Run 1

```
const out = arr.filter(x => x !== item);
```

Run 2

```
const i = arr.indexOf(item);  
arr.splice(i, 1);
```

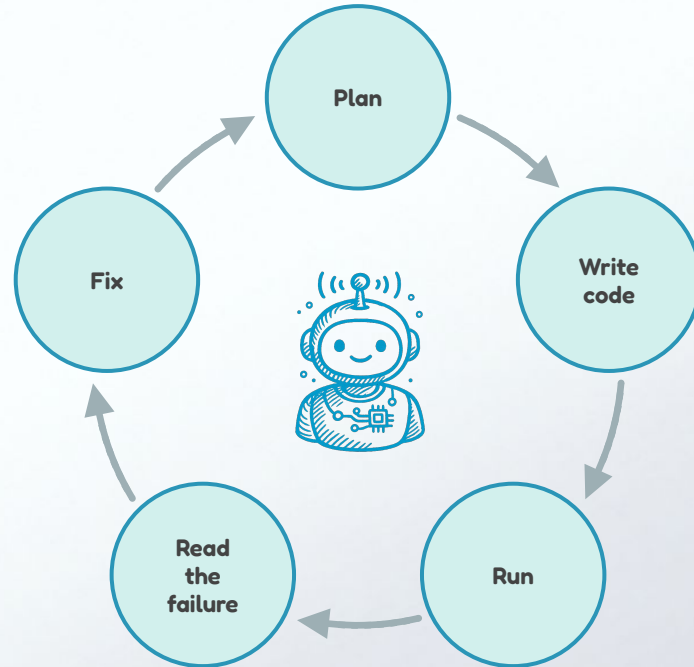
How a Developer Actually Works



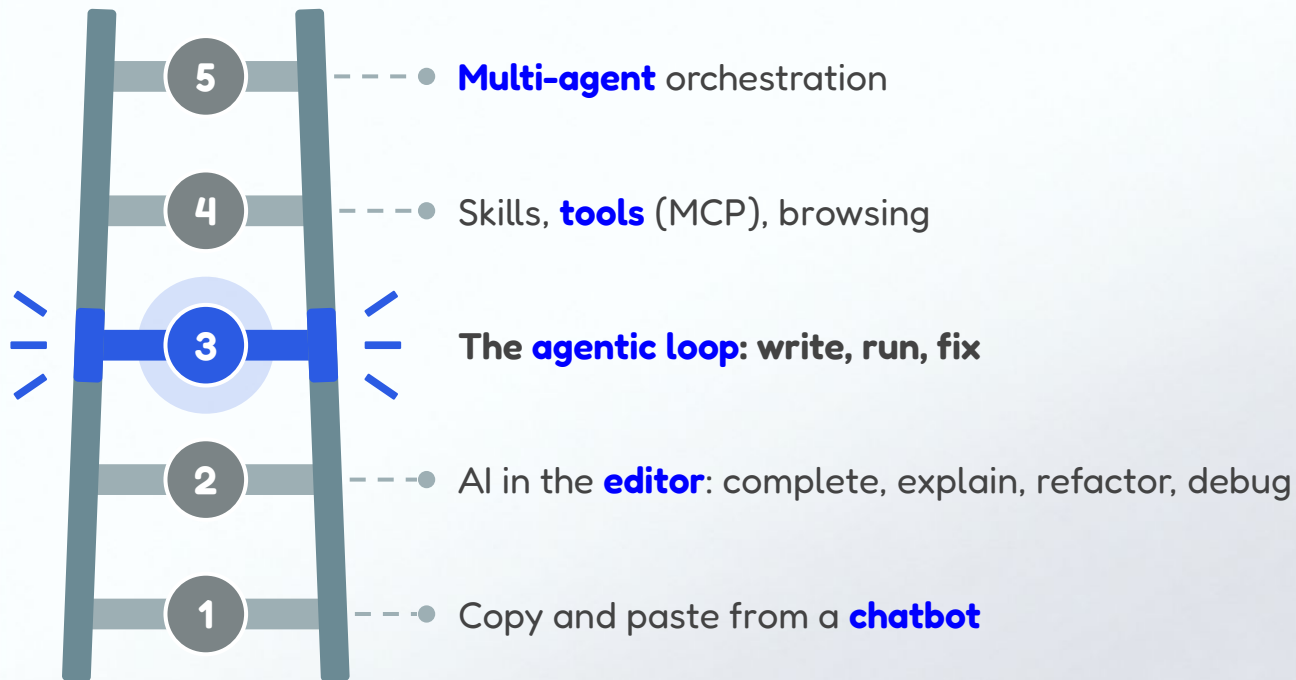


- It writes code
- It debugs its own code
- It sees what it rendered
- It builds the engineer

An Agent Runs the Same Loop



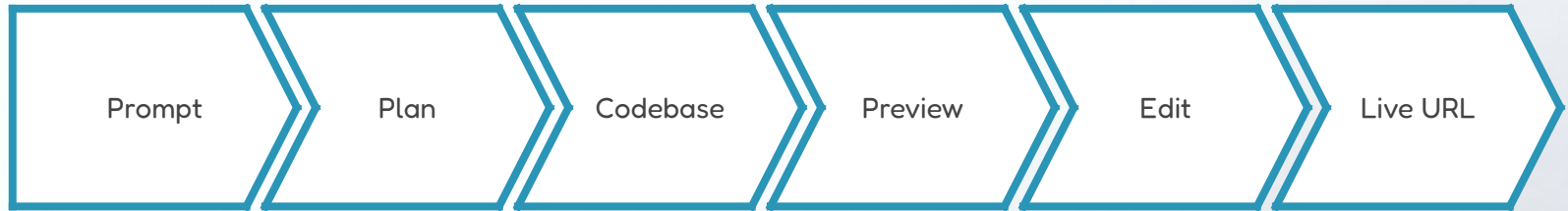
The Ladder



Ship Your First App



One Paragraph Becomes an App





38 Min

Your build block



1 Annotation

At least once



1 URL

Published, live

Build and Ship Your First App

- Lovable, or Purple to stay inside UW
- Come up with a cool mobile web app
- Iterate until it does what you want
- Use the annotation tool at least once

Before You Leave

- The published link, not the editor view
- A shared link to your Lovable project
- A screenshot of you annotating
- Purple: html file, prompts, screenshots

Due Tuesday, Oct 6, 11:59 pm

- Project 01: Prompt to Web App
- Week 1 skills survey
- Readings:
 - [Large Language Models from scratch](#)
 - [Large Language Models: Part 2](#)
 - [Welcome to Lovable](#)